

Spanner: Domain-Adapted Extractive Question Answering with BERT

Piyush Nirdesh Gothania

Abstract

We present *Spanner*, a compact toolkit for extractive question answering (QA) over user-supplied documents. Given a question and a passage, the system predicts the start and end of the passage span that answers it. Spanner fine-tunes a BERT model, previously trained on SQuAD 2.0, on a small set of domain questions, and handles documents longer than the model’s input limit with overlapping sliding windows. On a 17-question kangaroo knowledge set, domain fine-tuning raises Exact Match from 82.4 to 100.0. Every error made by the untuned model was a span-boundary mismatch, which fine-tuning corrected. The code, data and a command-line interface are available at <https://github.com/Anandpiyush21/spanner>.

1 Introduction

Extractive QA asks a model to locate the answer to a question inside a given passage, rather than generate free text. Because every answer is a substring of the source, the output can be traced back to the document and the model cannot invent facts. This makes it a good fit for question answering over manuals, notes and reference texts.

Pre-trained transformers such as BERT [1], fine-tuned on large benchmarks like SQuAD [2, 3], perform this task well on general text. A new domain, however, often has its own conventions for where an answer begins and ends. Spanner is a small, readable pipeline for adapting such a model to a domain with a few dozen examples, and then querying arbitrary documents from the command line.

2 Method

2.1 Task formulation

Let the question be q and the context be a token sequence $c = (c_1, \dots, c_n)$. The model receives the pair as

$$[\text{CLS}] \ q \ [\text{SEP}] \ c \ [\text{SEP}]$$

and produces, for every position i , a start logit s_i and an end logit e_i from a linear layer over BERT’s final hidden states. These are normalised with a softmax over the context tokens to give $P_{\text{start}}(i)$ and $P_{\text{end}}(j)$.

Training. Given gold token positions (i^*, j^*) , the loss is the mean of the two cross-entropies:

$$\mathcal{L} = -\frac{1}{2} (\log P_{\text{start}}(i^*) + \log P_{\text{end}}(j^*)).$$

Inference. The predicted answer is the highest-scoring valid span,

$$(\hat{i}, \hat{j}) = \arg \max_{i \leq j < i+L} P_{\text{start}}(i) P_{\text{end}}(j),$$

with maximum answer length $L = 30$ tokens. The span is mapped back to character offsets and sliced directly from the original text, which keeps its exact casing and punctuation.

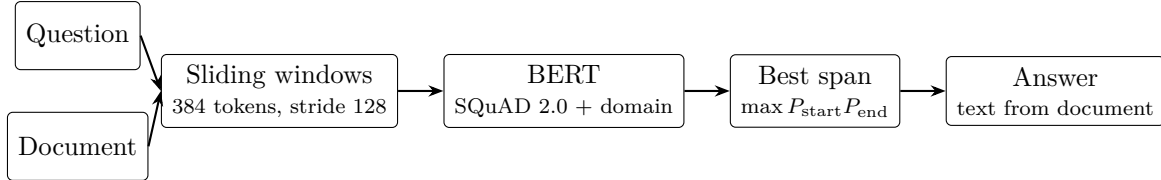


Figure 1: The Spanner inference pipeline.

2.2 Aligning answers to tokens

Annotations are given as character spans. We map them to token positions using the tokenizer’s offset mapping, which records the character range covered by each token: i^* is the first context token that ends after the answer starts, and j^* is the last context token that begins before the answer ends. The data loader rejects any example whose answer is not an exact substring of its context, so labels can never point at text that is not there.

2.3 Long documents

BERT accepts at most 512 tokens, and we use windows of 384. A longer context is split into overlapping windows, each repeating the question, with a stride of 128 tokens shared between consecutive windows, so that no answer is lost at a boundary (Figure 1). During training, a window that does not fully contain the answer is labelled with the [CLS] position, following the SQuAD 2.0 convention for “no answer”. At inference, every window is scored and the best span across all windows is returned.

We implement the windowing ourselves rather than using the tokenizer’s built-in overflow option, because `tokenizers 0.23.x` returns only a single overflow window for question–context pairs. The same function is used for training and inference, so both always see identical inputs.

3 Experimental setup

Data. The bundled dataset contains five passages about kangaroos (biology, behaviour, life cycle, ecology and conservation) with 17 questions in total, two to five per passage. Each answer is a short span copied verbatim from its passage. A separate, unannotated document, `kangaroo.txt`, is provided for interactive querying.

Model and training. We start from `deepset/bert-base-uncased-squad2`, a BERT-base model (110M parameters) fine-tuned on SQuAD 2.0, loaded with the Hugging Face Transformers library [4]. We train for 3 epochs with batch size 8 using AdamW [5] (learning rate 3×10^{-5} , weight decay 0.01), a linear schedule with 10% warmup, gradient clipping at 1.0, and seed 42. Training on a CPU takes about 90 seconds.

Metrics. We report the standard SQuAD metrics [2]. *Exact Match* (EM) is the percentage of predictions identical to the gold answer after normalisation (lower-casing and removing punctuation and articles). *F1* is the token-overlap F1 between prediction and gold answer, averaged over questions.

4 Results

The mean training loss fell from 0.234 in the first epoch to 0.158 and then 0.055. Table 1 compares the model before and after fine-tuning.

Model	EM	F1
SQuAD 2.0 BERT, no domain fine-tuning	82.4	97.1
+ 3 epochs on the kangaroo set (Spanner)	100.0	100.0

Table 1: Results on the 17-question kangaroo set.

Error analysis. The untuned model’s high F1 (97.1) shows that it already finds the right part of the passage. All three of its errors are boundary mismatches in which it omits a leading qualifier included in the gold answer:

Question	Predicted	Gold
How fast can a red kangaroo run?	56 kilometers per hour	<i>up to</i> 56 kilometers per hour
How long do joeys nurse?	an extended period	<i>for</i> an extended period
What does the digestive system allow?	extract nutrients ...	<i>efficiently</i> extract nutrients ...

Fine-tuning taught the model the annotation convention of this dataset: keep qualifying words such as *up to* as part of the answer.

On the separate `kangaroo.txt` document, which paraphrases the training passages, the fine-tuned model answers questions such as “*What do kangaroos eat?*” with “*grasses and other vegetation*” (confidence 0.98) and “*Where do female kangaroos nurture their young?*” with “*in a specialized pouch*” (0.98).

5 Limitations and future work

The scores in Table 1 are measured on the training questions, so they show that the model fits the domain’s conventions, not that it generalises to new questions. A held-out test set, ideally with several reference answers per question, is needed to measure generalisation. The dataset is also very small and covers a single topic. Natural next steps are to evaluate on a held-out split, to use the “no answer” option of SQuAD 2.0 so the system can decline unanswerable questions, and to add a retrieval stage so that questions can be asked over many documents at once.

6 Conclusion

Spanner shows that a SQuAD-trained BERT model can be adapted to a new domain’s answer conventions with only a handful of examples, provided the training labels are aligned precisely to tokens. Together with sliding-window inference over long documents, this gives a small and reliable way to ask questions about one’s own text.

References

- [1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, 2019.
- [2] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of EMNLP*, 2016.
- [3] P. Rajpurkar, R. Jia, and P. Liang. Know what you don’t know: Unanswerable questions for SQuAD. In *Proceedings of ACL*, 2018.
- [4] T. Wolf et al. Transformers: State-of-the-art natural language processing. In *Proceedings of EMNLP: System Demonstrations*, 2020.
- [5] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *Proceedings of ICLR*, 2019.